

RL-Course 2024/25: Final Project Report

PhiZero: Vojtěch Sýkora, Rojan Abolhassani, Swadesh Jana

February 27, 2025

1 Introduction

This report presents our solutions for the Hockey challenge undertaken as part of the final project of the lecture course Reinforcement Learning (RL) offered at the University of Tübingen.

Laser Hockey is a custom environment built using the Open AI gym. The environment is essentially a two-player hockey game, in which the agents compete to score a goal against each other. We train RL agents that can play in this environment and try to win in these games against not only the basic opponents provided but also other opponents.

The algorithms implemented are as follows:

1. Proximal Policy Optimization (PPO) - Vojtěch Sýkora
2. Soft Actor Critic (SAC) - Swadesh Jana
3. Modified Twin-Delayed DDPG (TD7) - Rojan Abolhassani

In the following sections we discuss the methods and experimental results in detail. The code has been uploaded to GitHub at <https://github.com/Swadesh13/rl-hockey>. Some parts of the code have been developed independently while some portions (such as a common HockeyAgent class) are developed and used by multiple members.

2 Proximal Policy Optimization (PPO)

2.1 PPO Method

PPO [10] is an on-policy actor-critic algorithm that optimizes a clipped surrogate objective to stabilize training.

It uses the likelihood ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ and clips it to prevent large policy updates:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right],$$

where $\hat{A}_t$ is the advantage function and ϵ is the clipping range. The full loss includes a value loss and entropy regularization:

$$L^{\text{PPO}}(\theta) = \hat{\mathbb{E}}_t [L^{\text{CLIP}}(\theta) - c_1(V_\theta(s_t) - V_t^{\text{target}})^2 + c_2 S(\pi_\theta(s_t))],$$

where c_1 and c_2 control the value loss and entropy terms.

2.1.1 Custom Rewards

One of our modifications to vanilla PPO is the inclusion of multiple specialized reward functions tailored to strategic behavior in a 2D hockey environment. Each individual reward function R_i is computed at every timestep based on the current state of the agent, puck, and environment. The final reward is then given by a weighted sum of original reward and all active additional rewards.

Since PPO involves multiple epochs of minibatch updates per rollout, the signals from R_{total} guide the gradient steps more consistently, reinforcing the targeted behaviors. Here we discuss the rewards individually even though we combined them in some experiments. In some rewards we use distances to reward or penalize agent movement which is a common approach in potential-based reward shaping [8]. The rewards are described in Table 1.

Reward	Formula and Description
Puck Proximity	$R_{\text{prox}} = \max(-0.02 d, -0.5)$, $d = \sqrt{(x_a - x_p)^2 + (y_a - y_p)^2}$ <i>Penalizes large distances of the agent from the puck.</i>
Intercept Path	$d_{\perp} = \frac{ m x_a - y_a + b }{\sqrt{m^2 + 1}}$, $R_{\text{intc}} = \max\left(-\frac{d_{\perp}}{8}, -0.5\right)$ $m = \frac{v_{py}}{v_{px}}$, $b = y_p - m x_p$ <i>Rewards alignment with the puck's predicted trajectory. [14]</i>
Blocking	$R_{\text{block}} = \begin{cases} 0.3, & \text{if } v_{px} < 0 \text{ and } \sqrt{(x_p - x_a)^2 + (y_p - y_a)^2} < 0.5, \\ 0, & \text{otherwise} \end{cases}$ <i>Gives a bonus for defensive interception near the agent's own goal.</i>
Puck Between Player And Goal	$R_{\text{between}} = \begin{cases} 0, & \text{if } x_a \leq x_p + \delta, \\ -0.5, & \text{otherwise} \end{cases}$ ($\delta \approx 0.2$) <i>Encourages positioning such that the puck stays between our agent and the opponent's goal.</i>
Offensive Pressure	$R_{\text{offense}} = \max(0.3 - 0.1 x_a - x_g , 0)$ <i>Rewards placing continuous pressure on the opponent's goal.</i>
Momentum Control	$v = \sqrt{v_{ax}^2 + v_{ay}^2}$, $\omega = v_{ang} $ $R_{\text{mom}} = \begin{cases} 0.3, & \text{if } 3 < v < 8 \text{ and } 1 < \omega < 4, \\ -0.02(v - 5)^2 - 0.02(\omega - 2)^2, & \text{otherwise} \end{cases}$ <i>Encourages controlled movement with moderate speeds. [13]</i>

Table 1:

Summary of Custom Reward Functions

Agent: a , Puck: p , Opponent's Goal: g , Puck Velocity in x -axis: v_{px}

2.1.2 Random Network Distillation (RND)

RND [1] provides an intrinsic reward that encourages exploration based on how well a *predictor network* can replicate the outputs of a frozen, randomly initialized *target network*. Let $f_{\text{target}}: \mathcal{S} \rightarrow \mathbb{R}^k$ map a state s to a k -dimensional feature space, and f_{pred} be a trainable network with the same architecture. At each timestep t , the intrinsic reward is computed as:

$$r_t^{\text{int}} = \|f_{\text{pred}}(s_{t+1}) - f_{\text{target}}(s_{t+1})\|^2. \quad (1)$$

The predictor f_{pred} is then updated by minimizing this mean squared error (MSE) over a minibatch of states encountered so far:

$$L_{\text{RND}} = \mathbb{E}[\|f_{\text{pred}}(s) - f_{\text{target}}(s)\|^2], \quad (2)$$

while the target network f_{target} remains fixed. States that are rare or novel produce a larger prediction error and thus a higher exploration bonus r_t^{int} .

We integrated RND into our PPO agent (Section 2.1) by, implementing both predictor and target networks as standard multi-layer perceptrons while keeping the target's parameters frozen throughout training. Secondly, following [1], each input observation s is normalized (whitened and clipped) before feeding it to both networks, ensuring stable scaling. Thirdly, after each batch of transitions, we compute the mean squared error loss (Eq. (2)) and update only f_{pred} . No gradients flow through f_{target} . Lastly, at every environment step, we compute r_t^{int} and combine it with the extrinsic reward r_t^{ext} using weights α and β : $R_t = \alpha r_t^{\text{ext}} + \beta r_t^{\text{int}}$. R_t is then passed to the PPO algorithm for advantage estimation and policy gradients.

2.2 PPO Experiments

For our experiments, we used the PPO implementation from `Stable-Baselines3` [9]. We chose the default `MlpPolicy`, which is a feed-forward architecture that outputs both (i) the mean and standard deviation

for a Gaussian action distribution (the actor) and (ii) a scalar value estimate (the critic). For a baseline we trained the agent against the `BasicOpponent`. We ran a grid search with over 300 different combinations of hyperparameters to obtain the final ones. The specific hyperparameters can be found in the repository’s `configs` folder. More information on experiments with noises in Appendix A.1.

2.2.1 Custom Reward Modifications

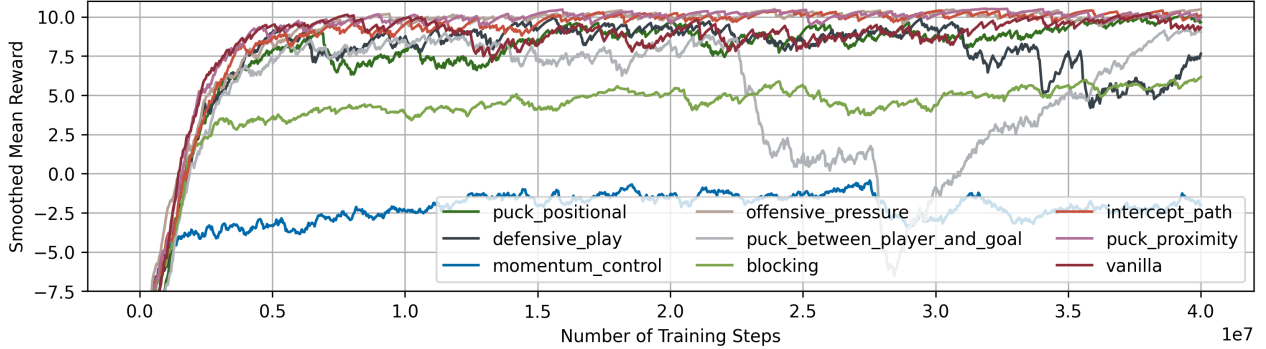


Figure 1: Effect of Different Rewards on PPO Performance

We tested these individual reward functions in the hockey environment and compared them to our baseline `vanilla` PPO. Each run trained for $\sim 4 \times 10^7$ timesteps against a basic strong opponent. Figure 1 shows the smoothed mean reward for each individual reward.

`Offensive Pressure (op)` and `Puck Proximity (pp)` achieved the highest average returns while also staying stable, suggesting that staying near the puck and pressing toward the opponent’s goal help win. Limiting the agent’s movement speed or enforcing defense led to worse results. For more analysis on the specific results of the rewards see Appendix A.2. We additionally explored combining the most promising rewards (`pp`, `op`, `ip`) and adding Gaussian noise. Interestingly, these combinations did not outperform simply using the `op` reward on its own. This result suggests that more reward terms can introduce conflicting objectives or excessive complexity in the agent’s learning signal.

2.2.2 Random Network Distillation (RND) Experiments and Results

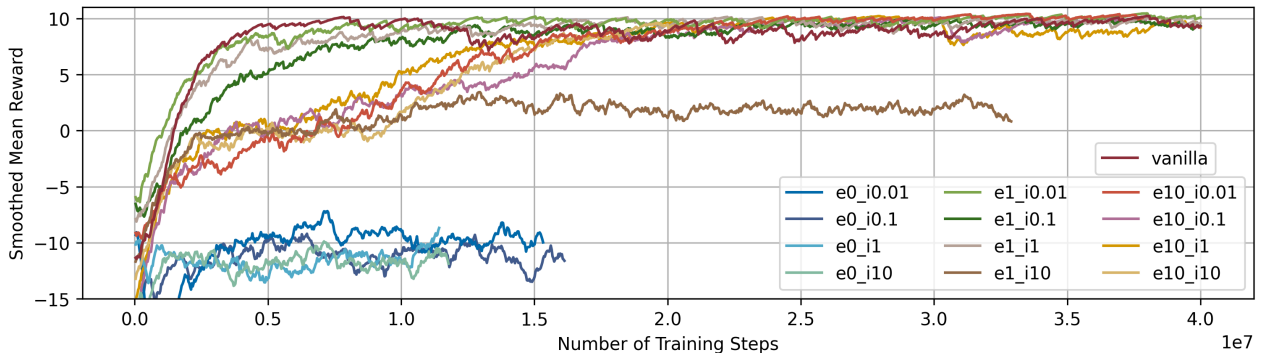


Figure 2: Effect of RND Extrinsic + Intrinsic reward combination on PPO Performance

Figure 2 shows selected training curves for different (α, β) pairs using 32 parallel environments. As can be seen, extrinsic reward had the biggest effect on training while intrinsic reward helped the agent perfect its skills through better exploration. $\alpha = 0$ (pure intrinsic reward) never yielded useful hockey skills. Quicker convergence was seen by $\alpha = 1$ compared to $\alpha = 10$, nevertheless, they both converged to a very good score after more than 20 million timesteps. Large β ($\beta = 1$ or 10) can overshadow extrinsic rewards if α is too

small, making the agent explore too much without regarding main environment objectives. On the other hand, small $\beta = 0.01$ often proved optimal across many runs since it provided a gentle bonus for novel states. The model we used in further training was $\alpha = 1, \beta = 0.01$ (labeled `e1_i0.01`) which converged quickly and achieved the highest final score overall while staying stable.

Scaling with Parallel Environments.

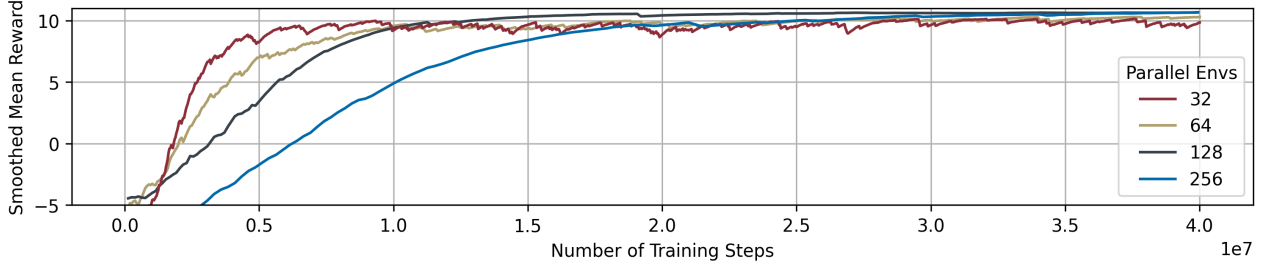


Figure 3: Effect of number of parallel environments PPO Performance

Figure 3 examines how RND performs with $\{32, 64, 128, 256\}$ parallel environments. Generally, more parallel environments yield broader state coverage and ultimately higher scores, however, they also require more computational resources. In some runs with 128 or 256 environments, the agent outperformed smaller-scale runs but only after a long training period ($>10M$ steps). We used 128 parallel environments in future training. Overall, these experiments confirm RND’s effectiveness in encouraging exploration within the hockey environment.

2.2.3 League & Self-Play

Once we had the best model for 6 different modifications and models for SAC and TD7, we utilized learning against these more advanced opponents. We experimented with training against 1, 2, 4, or 14 different opponents switching them randomly or using a specific decision function. We experimented with training against each opponent for varying blocks of 50k to 500k timesteps at a time. Unfortunately, even after a lot of training, these experiments did not yield good results, since the PPO agents were not able to consistently beat more than 3 of the total SAC and TD7 opponents. Nonetheless, it got better against variations of PPO and beat the basic opponents.

3 SAC

3.1 Method

Soft Actor-Critic (SAC) [5] is an off-policy, model-free reinforcement learning algorithm that optimizes for both reward maximization and entropy, encouraging exploration. SAC builds on the maximum entropy reinforcement learning framework, where the agent seeks to maximize the expected return while also maximizing entropy, leading to more robust policies.

Given policy π that produces action a_t at time t for a MDP with state s_t , the objective function of SAC is defined as:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t)))] \quad (3)$$

where r is the reward function, α is the temperature parameter that controls the trade-off between exploration (entropy maximization) and exploitation (reward maximization), and $\mathcal{H}(\pi(\cdot|s_t))$ represents the entropy of the policy.

Using inspiration from [7, 4], SAC maintains two Q-functions $Q_\theta(s, a)$ and selects the minimum for stability.

The soft Q-function update is given by:

$$J_Q(\theta) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim \mathcal{D}} \left[\frac{1}{2} \left(Q_\theta(s_t, a_t) - (r(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1} \sim \pi} [Q_{\bar{\theta}}(s_{t+1}, a_{t+1}) - \alpha \log \pi(a_{t+1} | s_{t+1})]) \right)^2 \right] \quad (4)$$

where $\mathcal{D}$ is the replay buffer, γ is the discount factor, s' is the next state, and $\bar{\theta}$ are the corresponding target networks. These target networks are updated (polyak update) after each iteration with parameter τ . Here, we deviate from the original implementation, where a third neural network estimated the value function.

The policy is parameterized as a Gaussian distribution, with the objective:

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim \mathcal{D}, a_t \sim \pi_\phi} [\alpha \log \pi_\phi(a_t | s_t) - Q_\theta(s_t, a_t)] \quad (5)$$

which encourages exploration by maximizing entropy.

Dynamic adjustment during training of the temperature parameter α [6] is also used. It minimizes:

$$J(\alpha) = \mathbb{E}_{a_t \sim \pi} [-\alpha (\log \pi(a_t | s_t) + H_0)] \quad (6)$$

where H_0 is a target entropy value, ensuring a balance between exploration and exploitation.

The algorithm is summarized below in Algorithm 1.

Algorithm 1 Soft Actor-Critic

```

1: Input:  $\theta_1, \theta_2, \phi$ 
2:  $\bar{\theta}_1 \leftarrow \theta_1, \bar{\theta}_2 \leftarrow \theta_2$  ▷ Initial parameters
3:  $\mathcal{D} \leftarrow \emptyset$  ▷ Initialize an empty replay pool
4: for each iteration do
5:   for each environment step do
6:      $a_t \sim \pi_\phi(a_t | s_t)$  ▷ Sample action from the policy
7:      $s_{t+1} \sim p(s_{t+1} | s_t, a_t)$  ▷ Sample transition from the environment
8:      $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r(s_t, a_t), s_{t+1})\}$  ▷ Store the transition in the replay pool
9:   end for
10:  for each gradient step do
11:     $\theta_i \leftarrow \theta_i - \lambda_Q \nabla_{\theta_i} J_Q(\theta_i)$  for  $i \in \{1, 2\}$  ▷ Update the Q-function parameters
12:     $\phi \leftarrow \phi - \lambda_\pi \nabla_\phi J_\pi(\phi)$  ▷ Update policy weights
13:     $\alpha \leftarrow \alpha - \lambda_\alpha \nabla_\alpha J(\alpha)$  ▷ Adjust temperature
14:     $\bar{\theta}_i \leftarrow \tau \theta_i + (1 - \tau) \bar{\theta}_i$  for  $i \in \{1, 2\}$  ▷ Update target network weights
15:  end for
16: end for
17: Output:  $\theta_1, \theta_2, \phi$  ▷ Optimized parameters

```

3.2 Experimental Details

The implementation of SAC has been adapted from `stable baselines3`. However, a few additional changes have been made that improve the algorithm performance. We keep the size of the policy and critic networks as 2 hidden layers with 256 nodes. The learning rate is kept at 10^{-3} with the Adam optimizer. Polyak update parameter $\tau = 0.005$, batch size of 1024 z' GPU, buffer size of 10^6 .

3.2.1 Noise Addition

Noise addition is important for exploration and stability. Since the policy itself is a Gaussian distribution (stochastic), noise is added to the output using the reparameterization "trick" or the reparameterization gradient estimator as:

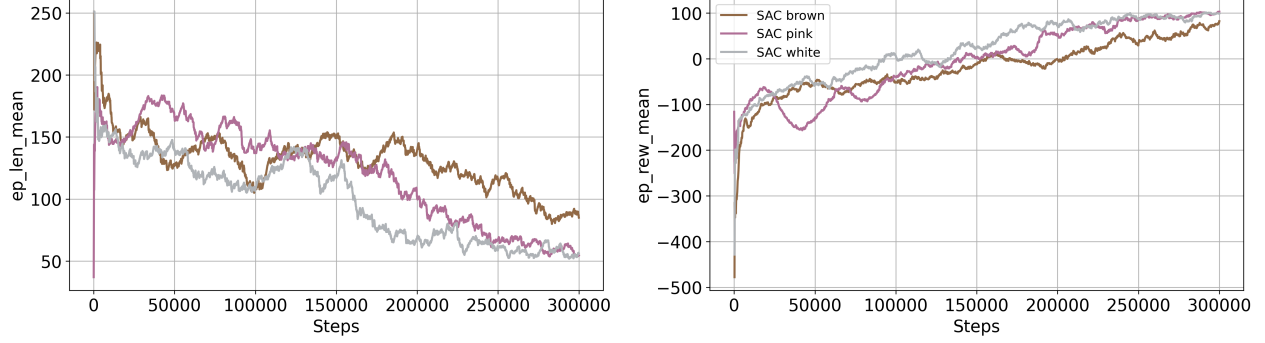


Figure 4: Episode length and total reward for SAC with brown, pink and white noises.

$$a_t = \mu_\phi(s_t) + \sigma_\phi(s_t) \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1)$$

where $\mu_\phi(s_t)$ and $\sigma_\phi(s_t)$ are the mean and standard deviation of the policy network. The reparameterization trick allows gradients to flow through sampled actions, improving stability.

Generally, white noise, i.e. random sampling from a uniform distribution, is used. These values are not correlated with time and can lead to very slow exploration. To improve this, we experiment with pink and brown noise. In general, colored noise with parameter β and signals $\epsilon(t)$ has the property

$$|\hat{\epsilon}(f)|^2 \propto f^{-\beta}$$

where $\hat{\epsilon}(f)$ is the Fourier transform of $\epsilon(t)$ with frequency f and $|\hat{\epsilon}(f)|^2$ is the power spectral density (PSD). For pink noise, we use $\beta = 1$ [2], while for brown noise, $\beta = 2$.

3.2.2 Replay Buffer

A simple improvement on top of the existing replay buffer helped the algorithm to learn the policy faster in the initial stages. An array named `experience` is introduced that keeps track of the number of times a particular sample is sampled randomly from the buffer. While ideally this should not be important, initially with very few samples and a high batch size, certain samples are overfitted more than others. Therefore, keeping track of the number of times a particular data was trained on helped. The next sampling used this information for sampling more of the less-sampled data.

Using a more specialized buffer such as prioritized replay buffer (sampling based on TD-errors) did not help in our experiments.

3.2.3 Reward shaping

To improve the model’s performance and have better more diverse data, a few models with additional rewards were designed. Other than the rewards discussed above, one special reward was to incentivize the model to shoot the puck at an angle such that the goal is made after one bounce off the surface. These however, are not part of the final algorithm and only help to create a more diverse and rich dataset. Additionally, adding the `touch_puck` reward and multiplying rewards by a factor of 10, contributed towards faster convergence

3.2.4 League & Self-play

A generalized league was created for the models to train on all other models and itself. It has multiple parameters that help to control the training and is used extensively to train the SAC model. Sampling opponents is usually done using a score function that keeps track of the current scores against other opponents and then samples an opponent based on either the lowest score or randomly.

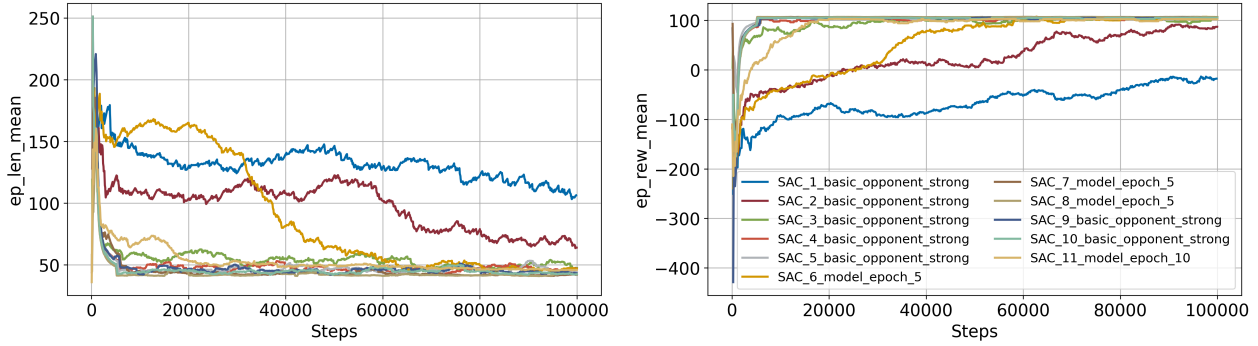


Figure 5: Episode length and total reward for SAC trained against basic opponent and itself (saved checkpoints at epoch 5 and 10). In the legend, the format for each colour is SAC_epoch_opponent.

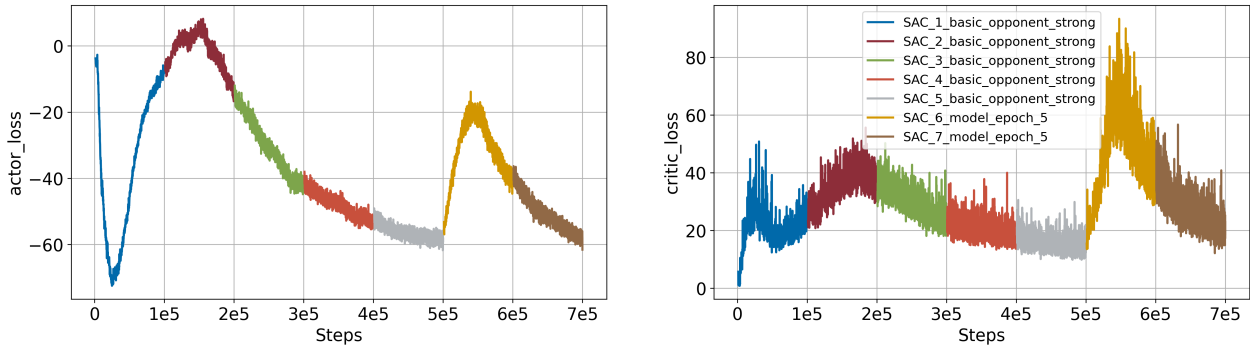


Figure 6: Actor and Critic losses for SAC with the same training configurations as Fig. 5.

3.3 Results & Discussion

The final model used for the competition was a result of multiple iterations of self-play and league-play. In this section, we show a sample league-play run, how the metrics look and comparison of different noises. Note that for rewards, y-axes are in magnitudes higher by factors of 10.

The comparison of the model learning with brown, pink and white noise has been shown until convergence in Fig. 4. We find that brown performs the worst while pink and white perform similarly. While white forms a seeming upperbound on pink, at the end, pink manages to converge faster or very similar to white. At the same time, pink shows a lot of variance, explained by its higher exploration power. Pink noise was the default choice while training the models.

The league training samples are shown in Fig. 5. As expected the initial training rewards are low and they slowly improve. In further epochs they converge very fast, until a new model is saved, such as its own version. Then the time to converge again increases, before converging faster. The loss curves are visualized in Fig. 6. Interestingly the actor loss decreases and then increases, in the same training iteration before slowly decreasing and stabilizing in further iterations. This happens until, again, a new model is used to train. The critic loss is much more unstable, and has a lot of fluctuations, but there is a similar trend of decreasing to convergence.

4 TD7

4.1 Method

TD7 [3] is a state-of-the-art offline and online off-policy reinforcement learning algorithm, built on TD3[4] with Loss-Adjusted Prioritized experience replay (LAP)[12], a behavior cloning term for offline reinforcement learning, State-Action Learned Embeddings (SALE), and policy checkpoints. Since our model is used for online training, we did not implement offline capabilities. Even though we implemented checkpointing

capabilities, early experiments indicated that the marginal benefits did not justify the performance overhead. Consequently, we decided against further exploration. For a detailed discussion on checkpointing, refer to Appendix B.2.

4.1.1 State-Action Learned Embeddings (SALE)

The objective of SALE is to discover learned embeddings z^{sa}, z^s which capture relevant structure in the observation space, as well as the transition dynamics of the environment. To do so, SALE utilizes a pair of encoders f, g , where $f(s)$ encodes the state s into the state embedding z^s , and $g(z^s, a)$ jointly encodes both state s and action a into the state-action embedding z^{sa} :

$$z^s := f(s), \quad z^{sa} := g(z^s, a). \quad (7)$$

The encoders are jointly trained using the mean squared error (MSE) between the state-action embedding z^{sa} and the embedding of the next state $z^{s'}$:

$$L(f, g) = (g(f(s), a) - |f(s')|_{\times})^2 = (z^{sa} - |z^{s'}|_{\times})^2. \quad (8)$$

where $|\cdot|_{\times}$ denotes the stop-gradient operation.

There are three additional considerations in how SALE is implemented in practice:

Normalized embeddings. The minimization of distances in embedding space can result in instability due to either monotonic growth or collapse to a redundant representation. To combat this risk, TD7 uses *AvgL1Norm*. Let x_i be the i -th dimension of an N -dimensional vector x , then the normalization is defined as:

$$\text{AvgL1Norm}(x) := \frac{x}{\frac{1}{N} \sum_i |x_i|}. \quad (9)$$

To maintain a consistent scale with the learned embeddings, we apply *AvgL1Norm* to the state and action inputs:

$$Q(z^{sa}, z^s, \text{AvgL1Norm}(\text{Linear}(s, a))), \quad \pi(z^s, \text{AvgL1Norm}(\text{Linear}(s))). \quad (10)$$

Fixed embeddings. Since an inconsistent input can cause instability, we freeze the embeddings used to train the current value and policy networks. This means that at iteration $t + 1$, the input to the current networks (Q_{t+1}, π_{t+1}) uses embeddings (z_t^{sa}, z_t^s) from the encoders (f_t, g_t) at the previous iteration t .

Clipped values. Extrapolation error occurs when deep value functions predict unrealistic values for state-action pairs that are rarely seen in the dataset. To mitigate this, clipping is applied to bound the value function's estimates within a stable range, preventing instability while maintaining learning efficiency.

$$Q_{t+1}(s, a) \approx r + \gamma \text{clip} \left(Q_t(s', a'), \min_{(s,a) \in D} Q_t(s, a), \max_{(s,a) \in D} Q_t(s, a) \right). \quad (11)$$

4.2 Loss-Adjusted Prioritized Experience Replay (LAP)

Experience is stored in a replay buffer and sampled using a prioritized scheme, where each transition tuple $i := (s, a, r, s')$ is sampled with probability:

$$p(i) = \frac{\max(|\delta(i)|^\alpha, 1)}{\sum_{j \in D} \max(|\delta(j)|^\alpha, 1)}, \quad \text{with } \delta(i) := Q(s, a) - y, \quad (12)$$

where y is the learning target.

4.2.1 Value Function

TD7 uses a pair of value functions (as motivated by TD3): $(Q_{t+1,1}, Q_{t+1,2})$, each taking input

$$[z_{t-1}^{s'a'}, z_{t-1}^{s'}, s', a'].$$

At each training step, both value functions are updated with the following loss:

$$L(Q_{t+1}) := \text{Huber}(\text{target} - Q_{t+1}(z_t^{sa}, z_t^s, s, a)), \quad (13)$$

where the target is defined as:

$$\text{target} := r + \gamma \text{clip}_{Q_{\min}, Q_{\max}}(\min(Q_{t,1}(x), Q_{t,2}(x))), \quad x := [z_{t-1}^{s'a'}, z_{t-1}^{s'}, s', a'], \quad (14)$$

$$a' := \pi_t(z_{t-1}^{s'}, s') + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \sigma^2), -c, c). \quad (15)$$

Taking the minimum of the value functions follows TD3's Clipped Double Q-learning (CDQ) [4]. The use of Huber loss aligns with TD3 and the LAP experience replay. a' is sampled and clipped in the same manner as TD3. The same embeddings (z_t^{sa}, z_t^s) are used for each value function.

$Q_{\min}$ and $Q_{\max}$ are updated at each time step:

$$\begin{aligned} Q_{\min} &\leftarrow \min(Q_{\min}, \text{target}), & \text{lower bound update} \\ Q_{\max} &\leftarrow \max(Q_{\max}, \text{target}), & \text{upper bound update} \end{aligned} \quad (16)$$

where the target is defined by Equation (14).

4.2.2 Policy

TD7 uses a single policy network with input $[z^s, s]$. The policy π_{t+1} is updated every second training step (following TD3's delayed updates) using the loss:

$$L(\pi_{t+1}) := -Q + \lambda |\mathbb{E}_{s \sim D}[Q]| \times (a_\pi - a)^2, \quad (17)$$

where $Q := 0.5(Q_{t+1,1}(x) + Q_{t+1,2}(x))$, with

$$x := [z_t^{sa_\pi}, z_t^s, s, a_\pi], \quad a_\pi := \pi_{t+1}(z_t^s, s). \quad (18)$$

The policy loss follows the deterministic policy gradient (DPG) [12], with an additional behavior cloning regularization term. The parameter λ is set to 0 for online reinforcement learning, which is always the case in our setting.

Every N steps (with a default value of 125), the networks are synchronized.

$$(Q_{t,1}, Q_{t,2}) \leftarrow (Q_{t+1,1}, Q_{t+1,2}), \quad \pi_t \leftarrow \pi_{t+1}, \quad (19)$$

$$(f_{t-1}, g_{t-1}) \leftarrow (f_t, g_t), \quad (f_t, g_t) \leftarrow (f_{t+1}, g_{t+1}). \quad (20)$$

4.2.3 Implementation Details

TD7 was implemented from scratch using the PyTorch library. The implementation was inspired by the original paper's code available in its GitHub repository. Several additional features, such as parallel training, integration with Hockey, and self-play were also implemented. Training was conducted in batches of 16, meaning that at each time steps, 16 parallel steps were executed across 16 different environments. These environments could feature different agents as opponents, ensuring diverse training interactions. For details regarding the parameters used in the experiment, please refer to Table 2 in Appendix B.3.

4.3 Experiments

Inspired by [11], we adopted self-play as a core component of our experiments. We aimed to replicate their success by finding stronger opponents for training.

We found that, given the structure of the tournament, training a model capable of competing against a wide variety of opponents was essential. The initial model, referred to as plain, was trained against a mixture of strong and weak basic opponents, with a higher proportion of strong opponents.

After this stage, we experimented with different custom reward functions and a variety of opponents, including different opponent compositions, previous learned models, and other variations. For rewards, we utilized Puck Proximity (PP) and Offensive Pressure (OP), as defined in Table 1.

Evaluation was performed every 10,000 training steps over 20 episodes against a strong opponent to assess performance. Strong opponents serve as a benchmark for detecting overfitting, especially when agents are trained primarily against themselves rather than strong opponents. Ideally, their performance against strong opponents should remain stable or improve over time. Figures 7 and 8 illustrate that our agents had no difficulty learning against strong opponents.

Our loss curves indicate that our models effectively learned both encoding and policy optimization. Both the encoder and actor loss show a smooth, gradual decrease, converging around 150k timesteps. In contrast, the critic loss exhibits more fluctuations and stabilizes later, around 240k timesteps.

To select the best model for the next training phase, we conducted a league-style evaluation, where models competed against each other. Figure 12 in Appendix B.1 presents the win percentages of the last eight models. The results clearly demonstrate that an agent trained against a specific opponent gains a significant advantage. Additionally, all models consistently outperformed the basic agents without difficulty.

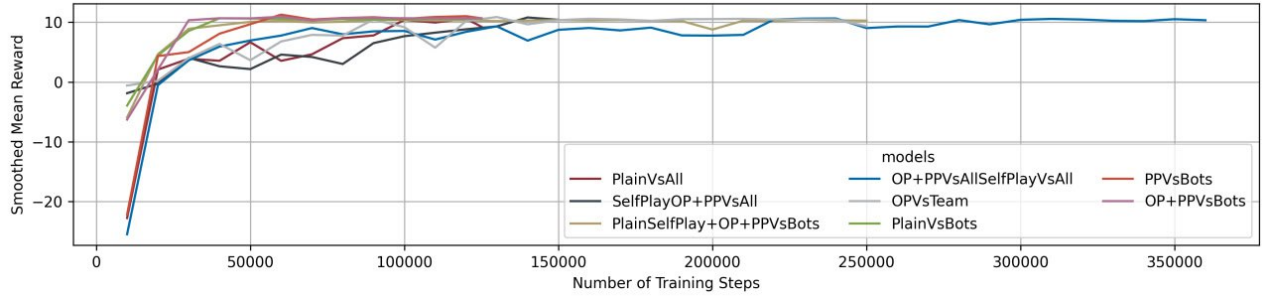


Figure 7: Average reward per episode collected every 10,000 training steps, evaluated against the strong basic opponent. Each evaluation consisted of 20 episodes, and the reported values represent the mean reward across these episodes.

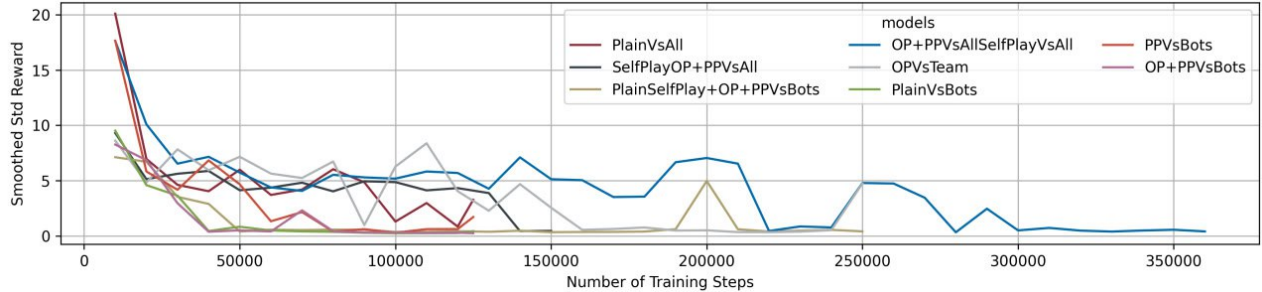


Figure 8: Standard deviation of reward per episode collected every 10,000 training steps, evaluated against the strong basic opponent. Each evaluation consisted of 20 episodes, and the reported values represent the mean reward across these episodes.

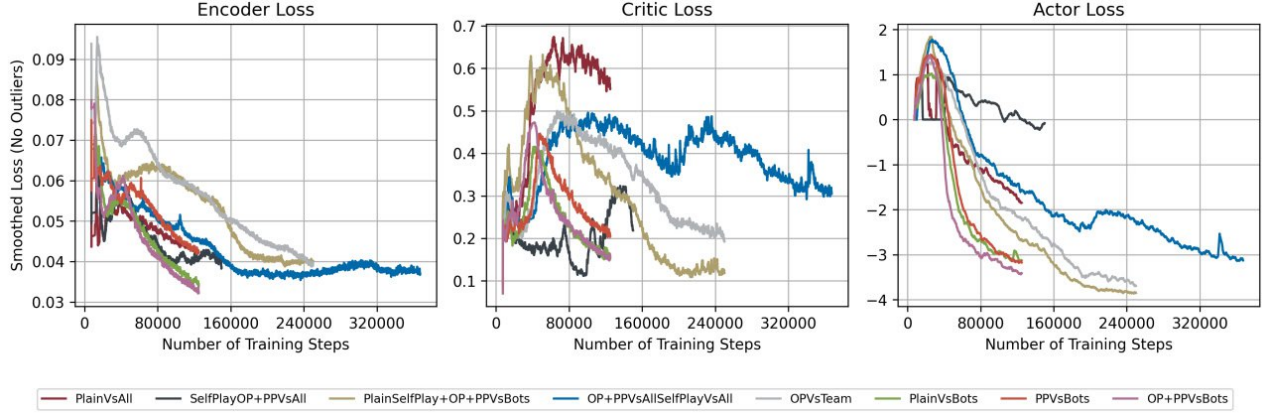


Figure 9: These losses were recorded in the middle of training. Outliers were removed using the Interquartile Range (IQR) method, and smoothing was applied using a moving average with a window size of 400.

5 Conclusion

In this work, three reinforcement learning algorithms, PPO, SAC, and TD7 were used to play in the Hockey environment. Each algorithm was modified for enhancing their performance.

PPO, an on-policy actor-critic method, was modified with custom reward functions and RND for improved exploration. Our results showed that strategic reward shaping, such as puck proximity and offensive pressure, significantly influenced agent performance. PPO benefited from Gaussian noise for stability but struggled against SAC or TD7 in self-play scenarios.

SAC, an off-policy algorithm, was adapted with pink noise to enhance exploration. Experimental results showed that pink noise accelerated convergence while maintaining high variance. SAC also used a simple modification of the replay buffer and some reward functions to improve sample diversity. League-play with self and opponents was important for training the algorithm.

TD7, a modified version of TD3, incorporated SALE and LAP to refine policy learning. While SALE helped capture meaningful state-action representations, LAP improved sample prioritization. Our self-play experiments revealed that TD7 adapted well to opponent variations, although checkpointing yielded minimal benefits in this scenario.

Self-play or league-play proved essential for training these algorithms effectively. Because each final model was trained against the current best opponent. Our focus remained on iterative improvement through model stacking rather than head-to-head evaluation.

6 Acknowledgment

We would like to thank Prof. Martius and the entire Autonomous Learning lab members for providing us with this exciting challenge. It was a fun and engaging learning experience that allowed us to explore and apply reinforcement learning techniques in a dynamic environment.

References

- [1] Y. Burda, H. Edwards, A. Storkey, and O. Klimov. Exploration by random network distillation. In *International Conference on Learning Representations (ICLR)*, 2019.
- [2] O. Eberhard, J. Hollenstein, C. Pinneri, and G. Martius. Pink noise is all you need: Colored noise exploration in deep reinforcement learning. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*, May 2023.
- [3] S. Fujimoto, W.-D. Chang, E. J. Smith, S. S. Gu, D. Precup, and D. Meger. For sale: State-action representation learning for deep reinforcement learning. 2023.
- [4] S. Fujimoto, H. Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*, pages 1587–1596. PMLR, 2018.
- [5] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018.
- [6] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.
- [7] H. Hasselt. Double q-learning. In J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta, editors, *Advances in Neural Information Processing Systems*, volume 23. Curran Associates, Inc., 2010.
- [8] A. Y. Ng, D. Harada, and S. J. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pages 278–287, 1999.
- [9] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, R. Ernestus, and N. Dormann. Stable-baselines3: Reliable reinforcement learning implementations, 2019. Software available from <https://github.com/DLR-RM/stable-baselines3>.
- [10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. *CoRR*, abs/1707.06347, 2017.
- [11] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.
- [12] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller. Deterministic policy gradient algorithms. In *Proceedings of the 31st International Conference on Machine Learning*, pages 387–395. PMLR, 2014.
- [13] M. W. Spong, S. Hutchinson, and M. Vidyasagar. *Robot Modeling and Control*. John Wiley & Sons, 2006.
- [14] E. W. Weisstein. Point-Line Distance (2-Dimensional). <https://mathworld.wolfram.com/Point-LineDistance2-Dimensional.html>, 2023. Accessed: 2025-02-22.

A PPO Appendix

A.1 PPO: Adding Noise

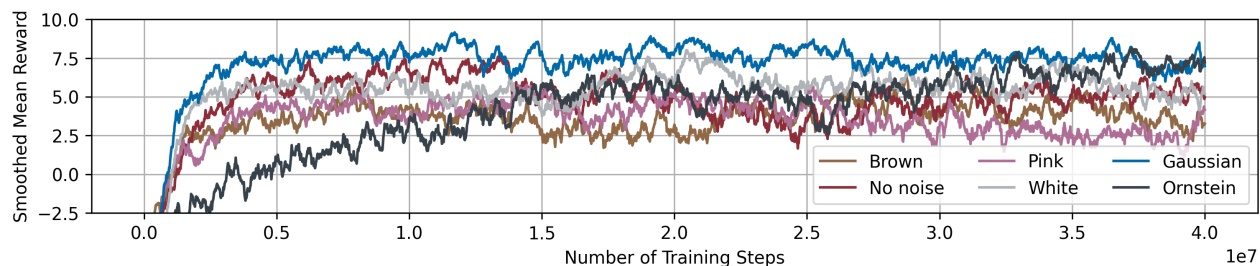


Figure 10: Effect of Different Noise Types on PPO Performance

Our first experiment with a modification to PPO was adding different noises. Pink, brown, and white noises [2] Gaussian noise [9] and Ornstein-Uhlenbeck Noise [9]. We had the baseline with no noise to identify if the added noises provided any meaningful improvement to exploration or stability. As can be seen in Figure 10, the Gaussian noise helped PPO the most. Gaussian noise added the right amount of randomness without destabilizing PPO’s learning. Since PPO is an on-policy method, the uncorrelatedness from Gaussian noise helps it explore different actions without getting stuck in a local noise pattern. It showed the fastest convergence while keeping its stability.

A.2 PPO: Custom Reward Experiments

We tested these individual reward functions in the hockey environment and compared them to our baseline vanilla PPO. Each run trained for $\sim 4 \times 10^7$ timesteps against a basic weak opponent. Figure 1 shows the smoothed mean reward for each individual reward.

Puck Proximity (pp) and **Offensive Pressure (op)** achieved the highest average returns while also staying stable, suggesting that staying near the puck and pressing toward the opponent’s goal help win.

Intercept Path (ip) also performed well, indicating that positioning itself to cut off the puck’s trajectory is advantageous.

Momentum Control performed the worst, never leaving the negatives. The underlying reward design penalizes high linear/angular velocities, but since the agent usually needs to move quickly to defend a goal or to juke the opponent, the agent’s behavior became too conservative and failed to outscore simpler methods.

Puck Between Player and Goal enforced a defensive positioning, however, we saw a very sharp performance drop. A reason for this might be because over-penalizing any deviation from staying behind the puck might remove good offensive positioning and the ability to adapt to dynamic puck movements like reflections from walls.

With **Blocking**, the agent learned to intercept the puck if it moved toward its own goal, but the score plateaued around 5. This is due to a similar reason to the previous reward. It focused too much on defense while neglecting offense.

Defensive Play and Puck Positional: These combined multiple individual rewards. They reached intermediate scores of around 8–9 but remained more unstable than top contenders like pp or op alone. Convex combinations of the individual rewards would be a great focus of further experiments.

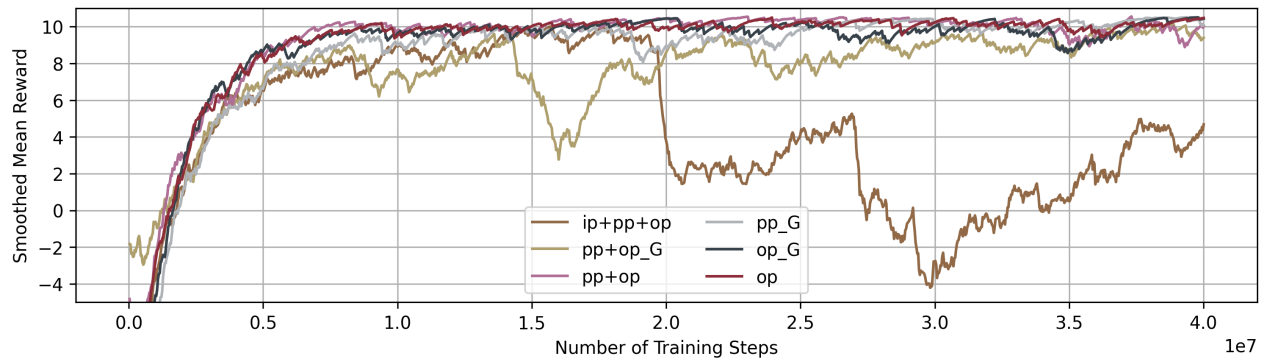


Figure 11: Effect of Multiple Reward and Noise Combination on PPO Performance

In Figure 11 We additionally explored combining the most promising rewards (`pp`, `op`, `ip`) and adding Gaussian noise (`G`). Interestingly, these combinations did not strictly outperform simply using the `op` reward on its own. This result suggests that more reward terms can introduce conflicting objectives or excessive complexity in the agent’s learning signal. Overall, the simplest individual rewards `offensive pressure` and `puck proximity` yielded the strongest and most stable improvements for our PPO agents in this environment.

B TD7 Appendix

B.1 Heatmap of TD7 Models Performance

This heatmap represents the results of a pair-matching league involving different TD7 agents, as well as both strong and basic agents. It is evident that rewards play a significant role compared to plain models. Additionally, an agent trained against another agent demonstrated strong dominance over them.

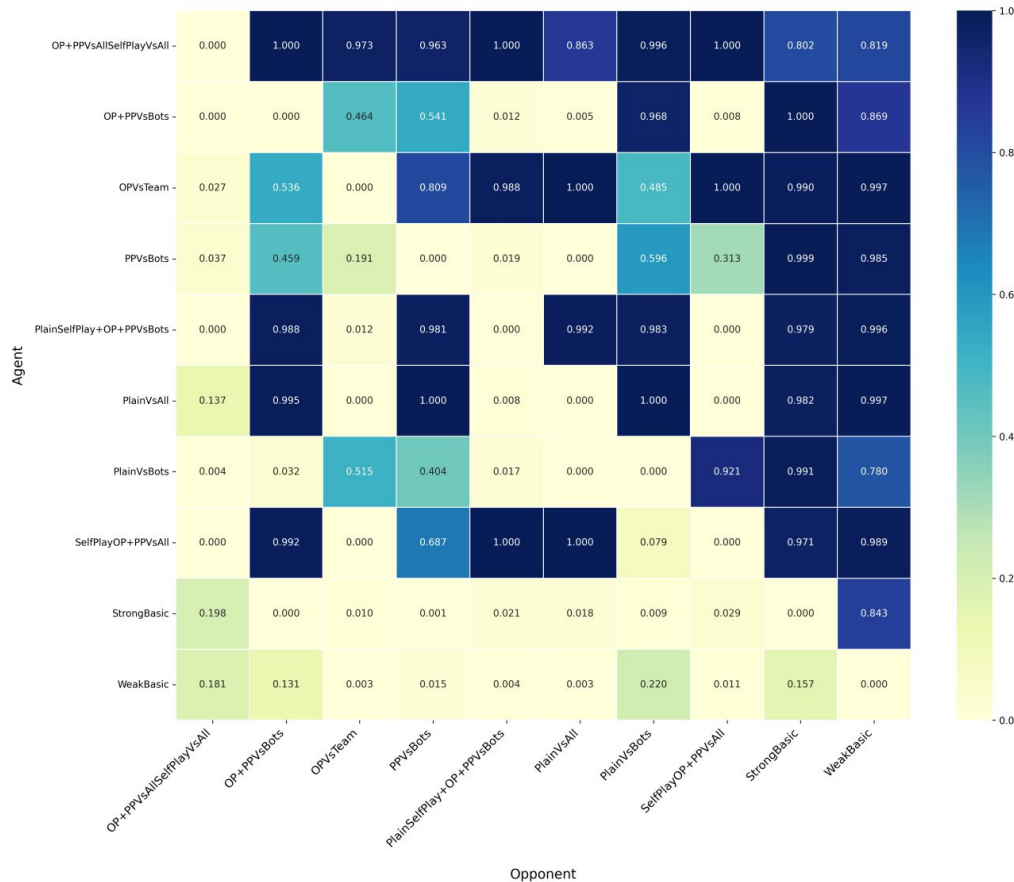


Figure 12: The win rate of different TD7 agents against each other, as well as against both basic and strong agents, was measured over 1,000 games.

B.2 TD7 Checkpointing

Deep Reinforcement Learning (RL) algorithms are notoriously unstable. The unreliable nature of deep RL algorithms suggests a need for stabilizing techniques. While we can often directly address the source of instability, some amount of instability is inherent to the combination of function approximation and RL. We propose the use of checkpoints to preserve evaluation performance, irrespective of the quality of the current learned policy.

A checkpoint is a snapshot of the parameters of a model, captured at a specific time during training. In supervised learning, checkpoints are often used to recall a previous set of high-performing parameters based on validation error and maintain a consistent performance across evaluations. Yet this technique is surprisingly absent from the deep RL toolkit for stabilizing policy performance.

In RL, using the checkpoint of a policy that obtained a high reward during training, instead of the current policy, could improve the stability of the performance at test time.

For off-policy deep RL algorithms, the standard training paradigm is to train after each time step (typically

at a one-to-one ratio: one gradient step for one data point). However, this means that the policy changes throughout each episode, making it hard to evaluate the performance. Similar to many on-policy algorithms, we propose to keep the policy fixed for several assessment episodes, then batch the training that would have occurred.

- **Standard off-policy RL:** Collect a data point $\rightarrow$ train once.
- **Proposed:** Collect N data points over several assessment episodes $\rightarrow$ train N times.

In a similar manner to evolutionary approaches, we can use these assessment episodes to judge if the current policy outperforms the previous best policy and checkpoint accordingly. At evaluation time, the checkpoint policy is used, rather than the current policy.

We make two additional modifications to this basic strategy.

B.2.1 Minimum Over Mean

Setting aside practical considerations, the optimal approach would be to evaluate the average performance of each policy using as many trials as possible. However, to preserve learning speed and sample efficiency, it is only sensible to use a handful of trials. As such, to penalize unstable policies using a finite number of assessment episodes, we use the minimum performance, rather than the mean performance. This approach also means that extra assessment episodes do not need to be wasted on poorly performing policies, since training can resume early if the performance of any episode falls below the checkpoint performance.

B.2.2 Variable Assessment Length

Freezing training for many episodes means that the environment is explored by a stale policy, reducing data diversity, and delaying feedback from policy updates. To counteract this effect, we restrict the number of assessment episodes used during the initial phase of training before increasing it.

B.3 TD7 Hyperparameters

TD7 Hyperparameters	
TD3	
Target policy noise σ	$\mathcal{N}(0, 0.22)$
Target policy noise clipping c	$(-0.5, 0.5)$
Policy update frequency	2
LAP	
Probability smoothing α	0.4
Minimum priority	1
Exploration	
Initial random exploration time steps	10k
Exploration noise	$\mathcal{N}(0, 0.1)$
Common	
Discount factor γ	0.99
Replay buffer capacity	2M
Mini-batch size	4096
Target update frequency	125
Optimizer	
Optimizer	Adam
Learning rate	3×10^{-4}

Table 2: TD7 Hyperparameters.